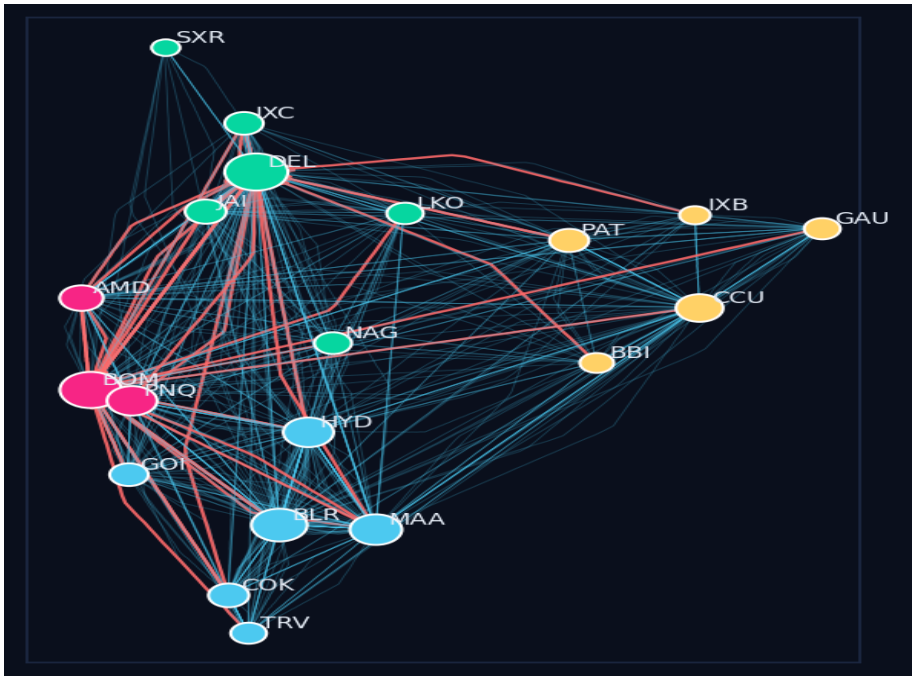
# SkyTrace

## Flight Trajectory Analysis using Spatio-Temporal Big Data Analytics

BDSMA Micro Project · B.E. Computer Engineering · Semester VII
SDG 9: Industry, Innovation & Infrastructure




Complete project guide: what we built, how every part works, how to run it, and exactly what to say on each slide.

# Contents


- 1. The project in one minute
- 2. The problem, with real examples
- 3. Why this is SDG 9
- 4. The dataset
- 5. The architecture: how data flows
- 6. Every technology explained simply, and what we did with it
- 7. Formulas and logic used
- 8. Results
- 9. The dashboard: what each tab shows
- 10. Use cases and benefits
- 11. How to run the project (college PC and Google Colab)
- 12. Folder structure: what each file does
- 13. Slide-by-slide presentation script
- 14. Viva questions and simple answers
- 15. Limitations and future scope

# 1. The project in one minute

Every passenger aeroplane carries a device called **ADS-B** (Automatic Dependent Surveillance–Broadcast). Every second it broadcasts the plane's **position (latitude, longitude)**, **height**, **speed** and **direction**. Anyone with a cheap receiver can pick these signals up, and websites like OpenSky Network collect them from all over the world.

If we join one plane's points in time order, we get its **trajectory**: the actual path it flew. **SkyTrace** collects lakhs of these points, stores them in **Hadoop**, processes them with **MapReduce** and **Spark**, saves the flight paths in **MongoDB**, and analyses the airport network with **graph algorithms** (Louvain, PageRank). The results appear on an interactive **web dashboard**.

**One-line explanation for anyone:** SkyTrace is like Google Maps traffic, but for the sky. It shows where and when Indian airspace is crowded, which flights waste distance and fuel, and which airports are the most important.

**Spatio-temporal** means we always look at two things together: **space** (where on the map) and **time** (what hour). Example question: *How many planes were in the box around Mumbai between 7 and 8 pm?*

# 2. The problem, with real examples

- **Crowded airports at rush hour.** Delhi and Mumbai are among the busiest airports in the world. Most flights want to depart or land at the same convenient times, early morning and evening.
- **Holding (planes circling).** When the runway is full, air traffic control tells arriving planes to fly in an oval racetrack pattern and wait their turn. Mumbai has crossing runways that cannot be used at the same time, so evening holding there is common. Delhi in winter fog has the same problem.
- **Bent routes.** Planes follow fixed air routes (airways) and military restricted areas, not straight lines. So a flight often covers more kilometres than the straight-line distance.
- **Cost.** An A320 burns about 2.4 kg of fuel per km in cruise and roughly 35–40 kg per minute in a holding pattern. 15 minutes of holding ≈ 500–600 kg of fuel, and every 1 kg of jet fuel produces about 3.16 kg of CO■.

**Real-world proof that this matters:** India built new airports at **Navi Mumbai** and **Noida (Jewar)** because Mumbai and Delhi ran out of runway capacity. Decisions like these need exactly the kind of data SkyTrace produces: where and when the airspace is full.

# 3. Why this is SDG 9

UN Sustainable Development Goal 9 is **Industry, Innovation and Infrastructure**. Target **9.1** asks for *quality, reliable, sustainable and resilient infrastructure*. Airports and the airspace above India are national infrastructure, just like highways and railways.

- **Infrastructure:** we find which airports and which hours are overloaded. That tells planners where to add a runway, a parallel taxiway or a better approach procedure.

- **Innovation:** we use modern big data tools (Hadoop, Spark, NoSQL, graph analytics) on open data.
- **Sustainability:** shorter routes and less holding mean less fuel and less CO■, which also supports SDG 13 (Climate Action).

# 4. The dataset

### *Source 1: OpenSky Network (real data)*

OpenSky Network (opensky-network.org) is a non-profit project run by universities. It publishes free historical ADS-B data as one CSV file per hour. Our Colab notebook downloads the hour **27 June 2022, 06:00 UTC** (11:30 am in India) and keeps only the points inside India's box (latitude 6°–37° N, longitude 68°–98° E).

### *Source 2: Labelled sample day (same format)*

Real OpenSky rows do **not** say where a flight came from or where it is going. Route analysis needs that, so we wrote `data/generate_dataset.py`. It creates one full 24-hour day in exactly the same column format for **20 major Indian airports**: **900 flights** and **67,279 position rows**. It copies real-world patterns:

- **Gravity model:** bigger airports get more flights, and nearby airports are connected more often (traffic $\propto$ size $\div$ distance$^{1.3}$).
- **Rush hours:** more departures 6–10 am and 5–9 pm IST.
- **Bent routes:** about 35% of flights take a dog-leg (a bend in the route) instead of the straight line.
- **Holding:** about 25% of flights arriving at Delhi and Mumbai circle before landing.
- **Realistic flight profile:** climb, cruise at standard flight levels, descend; one point per minute; small GPS noise.

**Be honest in the viva:** the route-efficiency and network numbers come from the sample day. The Hadoop, Spark and MongoDB code is written for the real OpenSky format and runs on the real hour too.

### *What one row looks like*

| Column | Meaning | Example |
|---|---|---|
| time | Unix timestamp (seconds since 1970) | 1719469800 |
| icao24 | Unique ID of the aircraft's transponder | 8056b8 |
| lat, lon | Position in degrees | 19.21, 72.95 |
| velocity | Ground speed in metres/second | 231.4 |
| heading | Direction in degrees (0 = North, 90 = East) | 184.0 |
| vertrate | Climb (+) or descent (–) rate in m/s | -6.2 |
| callsign | Flight number used on radio | AIC7713 |
| baroaltitude | Height in metres | 10050 |
| origin, destination | Airport codes (sample day only) | PNQ, BOM |

# 5. The architecture: how data flows

| Step | Tool | What happens | Syllabus |
|---|---|---|---|
| 1. Collect | OpenSky ADS-B | Download raw plane positions | - |
| 2. Store | Hadoop HDFS | Upload the big CSV; HDFS splits it into blocks across machines | Exp 1 |
| 3. Batch process | MapReduce (Hadoop Streaming, Python) | Job 1: count planes per map box per hour. Job 2: rebuild each flight and measure extra distance | Exp 3 |
| 4. Fast processing | Apache Spark | SQL queries, flight phases, holding detection, hotspots | Big data processing |
| 5. Store paths | MongoDB (NoSQL) | Each flight saved as a GeoJSON line with a geo index for location searches | Exp 2 |
| 6. Network analysis | NetworkX: Louvain, PageRank, Betweenness | Find airport groups and hub airports | Exp 6, 7 |
| 7. Show | HTML + Leaflet + Chart.js | Interactive dashboard with maps and charts | - |

# 6. Every technology explained simply

## 6.1 Hadoop HDFS (Experiment 1)

**What it is:** HDFS (Hadoop Distributed File System) stores very large files by cutting them into blocks (128 MB each by default) and spreading the blocks over many computers. Each block can be copied (replicated) so data survives if one computer fails.

**Simple example:** one big book torn into chapters, with each chapter kept in a different cupboard plus a photocopy somewhere else.

**Why we use it:** one day of global OpenSky data is about 1 GB, and a year is hundreds of GB. A normal laptop folder cannot store and process that efficiently.

**What we did:** in Colab we installed Hadoop 3.3.6 in pseudo-distributed mode (NameNode and DataNode on one machine) and ran:

```
hdfs dfs -mkdir -p /flights/raw /flights/archive      # make folders
hdfs dfs -put flights_states.csv /flights/raw/          # upload local -> HDFS
hdfs dfs -ls -h /flights/raw                            # list files
hdfs dfs -cat /flights/raw/flights_states.csv | head    # view
hdfs dfs -du -h /flights                                # size used
hdfs dfs -cp ... /flights/archive/                      # copy inside HDFS
hdfs dfs -mv ... opensky_backup.csv                     # move / rename
hdfs dfs -get /flights/archive/... ./from_hdfs.csv      # HDFS -> local
hdfs dfs -rm /flights/archive/opensky_backup.csv        # delete
hdfs fsck /flights/raw/flights_states.csv -files -blocks  # see blocks
```

## 6.2 MapReduce (Experiment 3)

**What it is:** a way to process huge data in parallel in two steps. **Map**: every machine reads its part and emits (key, value) pairs. **Shuffle/Sort**: Hadoop groups all pairs with the same key. **Reduce**: combine each group into an answer.

**Simple example:** election counting. Every booth counts its own votes (Map), votes are grouped by candidate (Shuffle), and the totals are added up (Reduce).

We wrote the jobs in Python and ran them with **Hadoop Streaming**, which lets any language read from stdin and write to stdout.

### *Job 1: spatio-temporal density grid (hadoop/mapper_grid.py + reducer_sum.py)*

- **Map:** for each row, find the 0.5° map box (about 55 km × 55 km), the hour, and the height band (GND / LOW <3 km / MID 3–8 km / HIGH >8 km). Emit `box_lat,box_lon,hour,band` → `1`.
- **Reduce:** add up the 1s for each key, which gives the number of plane reports in that box at that hour and height.
- **Use:** boxes with the biggest counts are **congestion hotspots**.

### *Job 2: trajectory reconstruction (hadoop/mapper_route.py + reducer_route.py)*

- **Map:** emit `icao24|callsign` → `time,lat,lon,alt,origin,destination`, so all points of one flight reach the same reducer.
- **Reduce:** sort that flight's points by time, add up the distance between consecutive points (flown distance), compare it with the straight great-circle distance, and output the extra %, duration and maximum height.
- **Use:** finds the most inefficient flights and routes.

```
Real output of Job 2
# flight -> from,to,flown_km,straight_km,extra%,minutes,max_height
8056b8|AIC7713   PNQ,BOM,349.8,123.9,182.40,43,7600
# Pune->Mumbai flew ~350 km for a 124 km trip
# because it circled (held) near Mumbai
```

## 6.3 Apache Spark

**What it is:** a faster successor to MapReduce. It keeps data in **memory (RAM)** instead of writing to disk after every step, and lets us write **SQL**. It reads directly from HDFS.

**Simple example:** MapReduce is like writing every rough calculation in a notebook and reading it back; Spark keeps the numbers in your head.

**What we did:**

- Counted active aircraft per hour with Spark SQL (`COUNT(DISTINCT icao24) GROUP BY hour`).
- **Window functions** (`LAG() OVER (PARTITION BY icao24 ORDER BY time)`) compare each point with the plane's previous point to get the turn angle.
- **Flight phase:** vertical rate > +1 m/s = Climb, < –1 = Descent, else Cruise; height < 30 m = Ground.
- **Holding detection:** below 3 km, if the plane's total turning adds up to ≥ 540° (one and a half full circles), it was holding.
- **Hotspots:** group airborne points into 0.5° boxes and take the top 10.

## 6.4 MongoDB, a NoSQL document store (Experiment 2)

**What it is:** a NoSQL database that stores flexible JSON-like **documents** instead of fixed tables. It has built-in support for **geographic data**.

**Why NoSQL here:** one flight path is a list of hundreds of points. In SQL that becomes hundreds of rows; in MongoDB it is one document. A **2dsphere index** lets us search by location quickly.

```
{ "_id": "8056b8", "callsign": "AIC7713", "origin": "PNQ", "destination": "BOM",
  "start": 1719469800, "end": 1719472380, "max_alt": 7600,
  "path": { "type": "LineString", "coordinates": [[73.92,18.58], [73.80,18.66], ...] } }
```

- **$near:** flights passing within 50 km of Nagpur (central India's airway crossroads).
- **$geoIntersects + time filter:** flights entering the Mumbai area polygon during the 5–9 pm rush. This is a true spatio-temporal query.
- **Aggregation pipeline:** the busiest routes ($group + $sort).

## 6.5 Graph analytics: Louvain, PageRank, Betweenness (Experiments 6 and 7)

We build a graph where each **airport is a node** and each **route is an edge**, weighted by the number of flights. This is the same kind of analysis used on social networks.

- **Louvain community detection:** groups nodes so there are many links inside a group and few between groups. It found **4 communities**: North (DEL, JAI, LKO, IXC, SXR, NAG), West (BOM, PNQ, AMD), South (BLR, MAA, HYD, COK, GOI, TRV) and East (CCU, GAU, PAT, BBI, IXB). Modularity Q = 0.234 (above 0 means real group structure).
- **PageRank:** the Google algorithm. An airport is important if it is connected to other important airports. Delhi and Mumbai rank highest.
- **Betweenness centrality:** how often an airport lies on the shortest path between two others. High betweenness means a transfer hub.
- **Social media link:** Louvain is like finding friend groups on Instagram, and PageRank is like finding influencers.

## 6.6 Python analytics pipeline and dashboard

`analytics/pipeline.py` runs the same logic as the Colab notebook using pandas and NetworkX, then writes `web/data/dashboard.json` (and dashboard.js). The dashboard (`web/index.html`) uses **Leaflet** for maps and **Chart.js** for charts. All libraries and the country outlines are bundled in `web/vendor`, so it also works offline. Online, a dark street map loads underneath.

## 6.7 Google Colab

A free cloud Jupyter notebook from Google. We use it because Hadoop, Spark and MongoDB need Linux and are hard to install on Windows. The notebook `notebook/Flight_Trajectory_Analysis.ipynb` installs everything and runs end to end.

# 7. Formulas and logic used

### *Haversine formula: distance between two points on Earth*

```
a = sin²(Δlat/2) + cos(lat1)·cos(lat2)·sin²(Δlon/2)
d = 2 · R · asin(√a)        R = 6371 km
```

### *Route inefficiency*

```
flown   = sum of haversine distances between consecutive points
straight = haversine(origin airport, destination airport)
inefficiency % = (flown / straight − 1) × 100
```

### *Avoidable CO■*

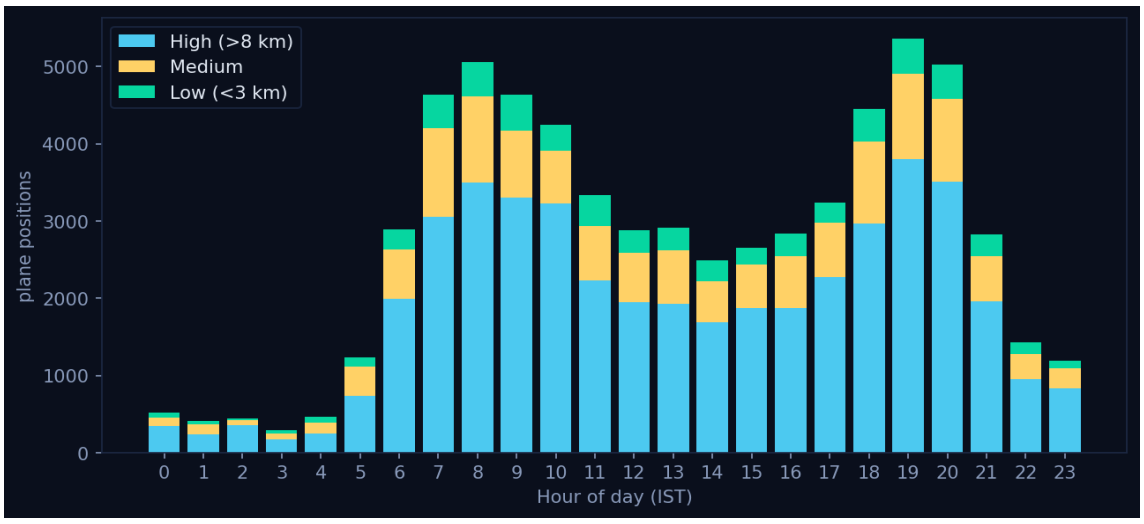
```
excess_km = flown − straight
CO■ (kg)  = excess_km × 2.4 kg fuel/km × 3.16 kg CO■ per kg fuel  ≈ excess_km × 7.6
```

### *Holding detection*

```
turn_i = ((heading_(i+1) − heading_i + 180) mod 360) − 180     # signed turn, −180..180
holding if |Σ turn_i over the approach segment| ≥ 540°        # 1.5 full circles
```

# 8. Results (sample day)

| Metric | Value | What it means |
|---|---|---|
| Position records | 67,279 | Rows of ADS-B data processed |
| Flights | 900 | Trajectories rebuilt by MapReduce Job 2 |
| Airports / routes | 20 / 235 | Nodes and directed links in the network |
| Average route inefficiency | 4.14% | Flights flew this much more than the straight line |
| Extra distance | 17,856 km / day | Total wasted flying |
| Avoidable CO■ | 135.7 t / day | ≈ 49,530 t per year |
| Holding events | 52 | All at DEL and BOM, during rush hours |
| Communities | 4 (Q = 0.234) | North, West, South, East regions |




*Traffic by hour of day: two clear rush-hour peaks.*

**Top 5 hotspot boxes** (nearest airport, reports): BOM (1048), PNQ (829), BLR (656), BOM (622), DEL (567).

# 9. The dashboard: what each tab shows

| Tab | What you see | What to point out |
|---|---|---|
| Top KPI cards | Key numbers | Size of the data and the CO■ that could be saved |
| Airspace Map | Every flight path coloured by height (green low, yellow medium, blue high, red = holding); heatmap; red hotspot boxes; hour slider; 'Holding only' | Move the slider to 8 and then 19; tick 'Holding only'; zoom into Mumbai to show the circles |
| Temporal Analysis | Stacked bar chart of traffic by hour and height; departures per hour; holding by airport | Morning and evening peaks; holding only at DEL and BOM |
| Route Efficiency | Worst routes, CO■ impact bars, table of the most inefficient flights | Pune–Mumbai is worst because of holding |
| Network & Communities | Airport graph on the map coloured by Louvain community, sized by PageRank; hub ranking table | 4 regional groups; DEL and BOM are the top hubs |
| Architecture | Pipeline steps and syllabus mapping | Which experiment each part covers |

# 10. Use cases and benefits

- **Airports Authority of India (AAI):** find detours and redesign routes (direct routing / Free Route Airspace), and check whether new procedures reduce holding.
- **Airport operators:** see which hours are saturated and plan runways, taxiways or new airports (as with Navi Mumbai and Noida).
- **Airlines:** move flights away from peak holding hours, plan fuel better, and reduce cost (fuel is about 30–40% of an airline's cost).
- **Government / DGCA:** measure aviation CO■ and track climate targets.
- **Passengers:** fewer delays and fewer missed connections.
- **Same method, other domains:** ship tracking (AIS), city bus GPS, delivery fleets, and drone traffic management.

# 11. How to run the project

## A. On any college PC (Windows) for the dashboard demo

- Install **Python 3.9+** from python.org (tick 'Add Python to PATH').
- Download the repo: github.com/manishraje28/SkyTrace → green **Code** button → **Download ZIP**, then extract it.
- Double-click **run_project.bat**. It installs packages, rebuilds the data and analytics, and opens `http://localhost:8765` in the browser.
- **No internet / no Python?** Just double-click `web/index.html`. The prebuilt data and all libraries are included, so it works offline (the street map background needs internet; the country outlines do not).
- The demo video is at `video/SkyTrace_Demo.mp4` as a backup.

## B. Hadoop / Spark / MongoDB in Google Colab

- Open colab.research.google.com → File → Upload notebook → choose `notebook/Flight_Trajectory_Analysis.ipynb`.
- Runtime → **Run all**. It takes about 6–8 minutes, mostly downloading Hadoop.
- Show these sections: HDFS commands, the two MapReduce jobs and their output, Spark SQL tables, MongoDB queries, Louvain communities, and the Folium map.
- **Tip:** run it once before the presentation so the outputs are already saved in the notebook.

## C. On Linux / Mac

```
chmod +x run_project.sh && ./run_project.sh
```

## D. Rebuild the PPT, the guide or the video

```
python ppt/build_ppt.py        # PPT with speaker notes
python docs/build_guide.py      # this PDF
python video/record.py          # demo video (needs playwright, edge-tts, ffmpeg)
```

# 12. Folder structure: what each file does

| Path | Purpose |
|---|---|
| run_project.bat / .sh | One-click setup and run |
| requirements.txt | Python packages needed |
| data/generate_dataset.py | Creates the 24-hour sample day in OpenSky format |
| data/flights_states.csv | The generated data (67k rows) |
| hadoop/mapper_grid.py, reducer_sum.py | MapReduce Job 1: density grid |
| hadoop/mapper_route.py, reducer_route.py | MapReduce Job 2: trajectory and route efficiency |
| analytics/pipeline.py | Local pipeline: phases, hotspots, routes, holding, graph → dashboard.json |

| Path | Purpose |
|---|---|
| notebook/Flight_Trajectory_Analysis.ipynb | Colab notebook: Hadoop, HDFS, MapReduce, Spark, MongoDB, Louvain |
| notebook/build_notebook.py | Script that generates the notebook |
| web/index.html | Dashboard |
| web/data/dashboard.json / .js | Analytics results used by the dashboard |
| web/vendor/ | Leaflet, Chart.js and country outlines (offline) |
| ppt/SkyTrace_Presentation.pptx | Presentation with speaker notes |
| ppt/content.py, build_ppt.py | Slide text and script; PPT builder |
| video/SkyTrace_Demo.mp4 | Narrated demo video |
| docs/SkyTrace_Project_Guide.pdf | This guide |

# 13. Slide-by-slide presentation script

Each slide below shows what is on the slide and what to say. The same script is also in the PowerPoint's speaker notes. Speak slowly; the whole talk takes about 7–8 minutes plus a 2–3 minute demo.

## Slide 1: SkyTrace

- Topic: Flight Trajectory Analysis (spatio-temporal analytics)
- Tools: Hadoop, MapReduce, Spark, MongoDB, Graph analysis
- Linked to UN SDG 9: Industry, Innovation & Infrastructure

> ***Say:*** *Good morning. Our micro project is called SkyTrace. In simple words, we take the location data that every aeroplane sends out, store it with big data tools like Hadoop, and study it to find where the sky over India is crowded, which flights take longer paths than needed, and which airports are the most important. This connects to the UN Sustainable Development Goal 9, which is about building good infrastructure.*

## Slide 2: What is this project?

- Every plane sends its position every few seconds, like a GPS ping
- Joining these points gives the plane's path, called a trajectory
- Spatio-temporal means we look at WHERE (place) and WHEN (time) together
- Think of it as Google Maps traffic, but for the sky

> ***Say:*** *Every commercial aeroplane has a device called ADS-B that broadcasts its location, height and speed every second. If we join these points, we get the path of the flight, which is called a trajectory. Spatio-temporal simply means we study place and time together, for example: how busy is the sky near Mumbai at 7 pm? You can imagine it like Google Maps showing red roads for traffic, but here we do it for the sky.*

## Slide 3: Why do we need this?

- Delhi and Mumbai airports are very busy in the morning and evening rush
- When the runway is full, planes circle in the air, called holding
- Planes also fly zig-zag airways instead of straight lines
- Result: extra fuel, more pollution, delays for passengers
- Example: 15 minutes of circling burns roughly 500-600 kg of fuel for an A320

***Say:*** *Here is the real problem. Indian airports like Delhi and Mumbai get very crowded at peak hours. When the runway is busy, air traffic control asks planes to fly in circles and wait. This is called holding. Also, planes do not always fly straight; they follow fixed air routes that bend. Both of these waste fuel, add pollution and delay passengers. For example, if an A320 circles for 15 minutes, it burns around 500 to 600 kilograms of extra fuel. Multiply this by hundreds of flights every day and the waste is very large.*

## Slide 4: Link to Sustainable Development Goal 9

- SDG 9 = build strong, reliable and sustainable infrastructure
- Airspace and airports are national infrastructure
- Our analysis shows WHERE new capacity is needed
- Shorter routes = less fuel = greener aviation (also helps SDG 13)

***Say:*** *SDG 9 asks countries to build infrastructure that is reliable and sustainable. Airports and the airspace above us are infrastructure, just like roads and railways. Our project gives data to decide where a new runway or a better route is needed. And when planes fly shorter routes, they burn less fuel, which also helps climate action, SDG 13.*

## Slide 5: The dataset

- Source: OpenSky Network, free real flight data used by researchers
- Each row = one plane at one moment: time, plane ID, latitude, longitude, height, speed, direction
- We use 1 real hour of data over India, plus a full 24-hour sample day
- Sample day: 20 airports, 900 flights, about 67,000 position records

***Say:*** *Our data comes from the OpenSky Network, a free website used by universities around the world to study flights. Each row tells us: at this time, this plane was at this latitude and longitude, at this height, flying at this speed and direction. In our Colab notebook we download one real hour of data over India. Real data does not tell where a flight started and ended, so we also made a sample day in the same format for 20 major Indian airports, with 900 flights and about 67,000 rows. It copies real patterns like the morning and evening rush.*

## Slide 6: How the system works

- 1. Collect: flight position data
- 2. Store: Hadoop HDFS, which splits big files across machines
- 3. Process: MapReduce and Spark crunch the numbers
- 4. Save: MongoDB stores each flight path for map searches
- 5. Analyse: graph of airports finds groups and hubs
- 6. Show: interactive web dashboard

> ***Say:*** *This is the flow of our system. First we collect the flight data. Then we store it in Hadoop HDFS, which can split a huge file into blocks and keep them on many computers. Next, MapReduce and Spark process the data in parallel. We store each flight's path in MongoDB, a NoSQL database that can search by location. Then we build a network of airports and analyse it. Finally, everything is shown on a web dashboard.*

## Slide 7: Hadoop and MapReduce

- HDFS commands used: put, ls, cat, cp, mv, get, rm
- Job 1: divide India's map into small boxes and count planes in each box per hour
- Job 2: join each plane's points in time order and measure the distance it really flew
- Compare with the straight-line distance to get the extra % flown
- Like election counting: many booths count at once, then totals are added

> ***Say:*** *In Hadoop we used the basic HDFS commands from our first experiment: put to upload, ls to list, cat to view, cp and mv to copy and move, get to download, and rm to delete. Then we wrote two MapReduce jobs. The first one divides the map of India into small boxes and counts how many planes were in each box every hour. That tells us the crowded areas. The second job collects all points of one plane, puts them in time order, and calculates how far it actually flew compared to the straight-line distance. MapReduce works like election counting: many booths count votes at the same time, which is the map step, and then all counts are added together, which is the reduce step.*

## Slide 8: Spark and MongoDB

- Spark reads data straight from HDFS and runs SQL queries fast, in memory
- Finds flight phase: climbing, cruising, descending
- Detects holding: a plane that turned more than 1.5 full circles near an airport
- MongoDB stores each flight path as a map line
- Example search: which flights passed within 50 km of Nagpur?

***Say:*** *Spark is faster than MapReduce because it keeps data in memory. We used Spark SQL to count active planes every hour and to label each point as climbing, cruising or descending. We also detect holding: if a plane keeps turning in the same direction for more than 540 degrees, that means one and a half circles, so it is waiting to land. In MongoDB, every flight is saved as a line on the map with a special geo index. That lets us ask questions like: which flights passed within 50 kilometres of Nagpur, or which flights entered the Mumbai area during the evening rush.*

## Slide 9: Airport network: groups and hubs

- Airports = dots, flights between them = lines
- Louvain algorithm finds groups of airports that fly mostly among themselves
- Found 4 regions: North, West, South, East
- PageRank finds the most important hubs: Delhi and Mumbai
- Like finding friend groups and influencers on Instagram

***Say:*** *Next we turned the airports into a network. Each airport is a dot and each route is a line. We used the Louvain community detection algorithm from our syllabus. It found four natural groups: North, West, South and East India. Then we used PageRank and betweenness to find the most important airports, which are Delhi and Mumbai. This is exactly like social media analysis: Louvain finds friend groups, and PageRank finds influencers. Here the influencers are hub airports.*

## Slide 10: What we found

- Flights flew 4.1% more distance than the straight path, on average
- About 17,900 km of extra flying in one day
- About 136 tonnes of CO2 that could be avoided every day
- 52 planes had to circle (hold), all at Delhi and Mumbai, at rush hours
- Busiest hours: 6-10 am and 5-9 pm

***Say:*** *These are our results for the sample day. On average, flights flew 4.1 percent more than the straight path. That adds up to nearly 18,000 kilometres of extra flying in one day, which is about 136 tonnes of carbon dioxide that could be saved. 52 planes had to circle before landing, all at Delhi and Mumbai, and all during the morning and evening rush. This clearly shows where and when the infrastructure is under pressure.*

## Slide 11: The dashboard

- Airspace map: every flight path coloured by height, with a time slider
- Heatmap and hotspot boxes show crowded areas
- Temporal tab: traffic by hour, holding by airport
- Route tab: worst routes and CO2 impact
- Network tab: airport groups and hub ranking

***Say:*** *Now let me show the dashboard. On the first tab you see every flight path over India. Green is low, yellow is medium and blue is high altitude. Red paths are planes that circled. I can move this slider to see flights at any hour. The temporal tab shows the morning and evening peaks. The route tab lists the routes with the most extra distance and the CO2 impact. The network tab shows the airport groups found by Louvain and the ranking of hub airports.*

## Slide 12: Who can use this and how

- Airports Authority of India: plan new routes and airspace changes
- Airport operators: decide when a new runway or terminal is needed
- Airlines: choose better routes and departure times to save fuel
- Government: track aviation pollution against climate targets
- Passengers: fewer delays, more on-time flights

***Say:*** *Who benefits from this? The Airports Authority of India can use it to plan direct routes and redesign busy airspace. Airport operators can see when the runway is full and plan new capacity, like the new Navi Mumbai and Noida airports, which were built because the old ones are crowded. Airlines can schedule flights away from rush hours and save fuel. The government can track pollution, and in the end passengers get fewer delays.*

## Slide 13: Conclusion and future scope

- Built a complete big data pipeline: Hadoop, MapReduce, Spark, MongoDB, Graphs
- Turned raw flight points into useful decisions for infrastructure
- Future: live data with Kafka, predict delays with machine learning, add weather data
- Thank you!

***Say:*** *To conclude, we built a full big data pipeline using the tools from our syllabus and used it to turn raw flight points into useful insights for infrastructure planning. In future we can connect live data using Kafka, predict delays using machine learning, and add weather data, because fog in Delhi is a major cause of delays. Thank you, we are happy to take questions.*

# 14. Viva questions and simple answers

**Q. What is a trajectory?**

The path of a moving object over time: a list of (time, latitude, longitude, height) points for one flight.

**Q. What does spatio-temporal mean?**

Data that has both a location (space) and a time. We analyse both together, for example the number of planes in a map box during a given hour.

**Q. What is ADS-B?**

A system where aircraft broadcast their GPS position, height and speed about every second. Ground receivers and OpenSky collect it.

**Q. Why Hadoop when the file is small?**

The sample is small so it runs in Colab, but real ADS-B data is about 1 GB per day globally and hundreds of GB per year. The same code scales by adding nodes; nothing changes in the logic.

**Q. What is the difference between HDFS and a normal file system?**

HDFS splits files into large blocks spread over many machines with replication (default 3 copies), so it can store huge files and survive machine failure. A normal file system lives on one disk.

**Q. Explain Map and Reduce in your project.**

In Job 1, Map turns each point into (map box, hour, height band) $\rightarrow$ 1, and Reduce adds the 1s to get counts per box. In Job 2, Map groups points by flight, and Reduce sorts them by time and calculates the distance flown.

**Q. Why Spark if we have MapReduce?**

Spark keeps data in memory, so it is much faster for multi-step work, and it supports SQL and window functions, which we need to compare each point with the previous one.

**Q. Why MongoDB and not MySQL?**

One flight path is a variable-length list of points. MongoDB stores it as one GeoJSON document and its 2dsphere index supports location queries like $near and $geoIntersects.

**Q. What is Louvain?**

A community detection algorithm. It moves nodes between groups to maximise modularity: many links inside groups and few between them. It found 4 regional airport groups.

**Q. What is modularity?**

A score from about –0.5 to 1 that measures how good a grouping is. Above about 0.2 means a clear group structure; ours is 0.23.

**Q. What is PageRank?**

Google's algorithm: a node is important if important nodes link to it. For airports, it identifies the hubs (Delhi, Mumbai).

**Q. How do you detect holding?**

We add up a plane's turn angles near the airport. If it turned 540° or more in one direction (1.5 circles), it was circling and waiting.

**Q. How did you calculate CO■?**

Extra km × 2.4 kg fuel per km (A320 cruise) × 3.16 kg CO■ per kg of jet fuel, which is about 7.6 kg CO■ per extra km.

**Q. Is the data real?**

The Colab notebook uses one real hour from OpenSky. Real data has no origin or destination, so route and network analysis use a realistic simulated day in the same format. We say this openly.

**Q. How is this related to SDG 9?**

SDG 9 is about reliable and sustainable infrastructure. Airspace and airports are infrastructure; we show where capacity runs out and how much fuel and CO■ inefficiency costs, which supports planning decisions.

**Q. How is this related to social media analytics?**

The airport network is analysed with the same structure-based methods used for social networks: community detection (Louvain) and influencer ranking (PageRank, betweenness).

**Q. What is great-circle distance?**

The shortest distance between two points on a sphere (the Earth). We calculate it with the haversine formula.

# 15. Limitations and future scope

- **Sample data:** route numbers come from a simulated day; next step is matching real OpenSky flights to airports by their first and last positions.
- **Single-node Hadoop:** Colab runs Hadoop on one machine; a real deployment would use a multi-node cluster or cloud (AWS EMR, Google Dataproc).
- **Future:** live streaming with Kafka + Spark Structured Streaming; delay prediction with machine learning; weather data (Delhi fog, monsoon); 3D visualisation of altitude; runway capacity simulation.